CS505 HW0: Classification - Written Report

Author: Zhengzheng Tang
BUID: U07312313
Date: February 2026

Task 1: Bag-of-words Classification

Q1 (3 points)

Implementation: The `BoWFeaturizer` class was implemented in `models.py`. The `build_vocab` method tokenizes text using NLTK's `word_tokenize`, converts to lowercase, counts token frequencies, and keeps the top 5,000 most frequent tokens. The `get_feature_vector` method returns a count vector of vocabulary tokens present in the input text.

Results using `python lang_classifier.py --model BOW`:

Metric	Value
Train Accuracy	0.9782
Dev Accuracy	0.7459
Time	~6 seconds

Q2 (1 point)

Why are training and development accuracies different?

The training accuracy (0.9782) is significantly higher than the development accuracy (0.7459) due to **overfitting**. The model memorizes patterns specific to the training data, including noise and idiosyncrasies that don't generalize to unseen examples. The development set contains different examples that the model hasn't seen during training, so it performs worse.

Q3 (2 points)

Top-5 weight indices and corresponding tokens for each class:

After training the BOW model, I examined the weight matrix (shape: 4 x V) to find the highest-weighted tokens for each class:

Class	Label	Top-5 Indices	Top-5 Tokens
0	World	25, 8, 82, 52, 48	'...', '-', 'afp', 'minister', 'president'
1	Sports	236, 443, 88, 232, 180	'team', 'saturday', 'sunday', 'night', 'olympic'
2	Business	89, 60, 320, 15, 81	'company', 'oil', 'business', 'reuters', 'inc.'
3	Tech/Science	34, 301, 331, 463, 545	'ap', 'scientists', 'space', 'space.com', 'nasa'

Analysis:

- **World news** features political terms ('minister', 'president') and news agency identifier ('afp' - Agence France-Presse). The punctuation tokens ('...', '-') may appear frequently in international news formatting.
- **Sports** contains time-related words ('saturday', 'sunday', 'night') indicating game schedules, and 'team' and 'olympic' are directly sports-related.
- **Business** shows corporate terms ('company', 'inc.') and economic indicators ('oil', 'business'), plus 'reuters' as a major financial news source.
- **Tech/Science** includes scientific terms ('scientists', 'space') and domain-specific sources ('space.com', 'nasa', 'ap').

The most important tokens for each class are reasonable and correspond to the domain vocabulary of each news category.

Task 2: Logistic Regression

Q4 (10 points)

Implementation: The `LogisticRegressionClassifier` class was implemented with:

- `forward`: Computes z = W^T * x + b using `torch.matmul`
- `softmax`: Implements numerically stable softmax by subtracting max before exponentiating
- `train_logistic_regression`: Implements SGD with manual gradient computation

Results using `python lang_classifier.py --model LR`:

Metric	Value
Train Accuracy	0.9770
Dev Accuracy	0.7471
Time	~6 seconds

Q5 (3 points)

Hyperparameter Tuning Results:

Note: The training data contains 8,876 unique tokens total, so any vocab_size above this effectively uses the full vocabulary.

vocab_size	lr	epochs	Train Accuracy	Dev Accuracy
5000	0.01	10	0.9770	0.7471
5000	0.001	10	0.8194	0.7035
5000	0.1	10	0.9994	0.7576
5000	0.01	20	0.9903	0.7541
5000	0.01	30	0.9976	0.7553
8876	0.01	10	0.9473	0.7459
8876	0.01	20	0.9976	0.7518

Best configuration: vocab_size=5000, lr=0.1, epochs=10, Train Accuracy=0.9994, Dev Accuracy=0.7576

Learning Rate Analysis:

- **Too high (lr=0.1):** The model converges very quickly (loss drops to 0.02 by epoch 10), achieving near-perfect train accuracy (0.9994). However, the loss fluctuates between epochs because large gradient steps overshoot the optimal parameters, causing the optimization to bounce around the minimum. Despite this, the final dev accuracy (0.7576) is slightly higher than the baseline, suggesting the rapid convergence found a reasonable solution before overfitting too severely.
- **Too low (lr=0.001):** The model converges very slowly — after 10 epochs the loss is still at 0.56 (compared to 0.19 for lr=0.01). The train accuracy is only 0.8194 and dev accuracy drops to 0.7035 because the small gradient updates are insufficient to learn the decision boundary properly within the limited number of epochs.

Task 3: Feature Engineering

Q6a (3 points)

Bigram Implementation: The `BigramFeaturizer` class generates consecutive word pairs (e.g., "the|cat") and builds a vocabulary of the most frequent bigrams (vocab_size=20,000).

Results using `python lang_classifier.py --model BIGRAM`:

Metric	Value
Train Accuracy	0.9533
Dev Accuracy	0.6729

Q6b (2 points)

Why does the bigram model perform worse despite being more expressive?

The bigram model suffers from **feature sparsity** leading to **overfitting**. While unigrams have ~5K unique tokens, bigrams have potentially millions of combinations, but most bigrams appear only once or twice in the training data. With only 1,650 training examples, most bigram features have extremely sparse observations, so the model learns weights for bigrams that happened to appear in training but won't generalize. The high train accuracy (0.9533) combined with lower dev accuracy (0.6729) confirms this overfitting pattern. As discussed in the textbook (Chapter 3.5), more complex models with more parameters require more training data to estimate reliably; with insufficient data, simpler models (unigrams) generalize better.

Q7a (8 points)

Custom Feature 1: TF-IDF Featurizer

Location: `models.py`, class `TFIDFFeaturizer`

Description: TF-IDF (Term Frequency-Inverse Document Frequency) weighs terms by their importance. Common words get lower weights (low IDF), while rare but discriminative words get higher weights.

Implementation details:

- TF = count(word) / total_words_in_document
- IDF = log(N / (df + 1)) + 1 (smoothed)
- Final vector is L2 normalized

Results (vocab_size=5000, lr=0.1, epochs=30):

Metric	Value
Train Accuracy	0.9933
Dev Accuracy	0.7306
Dev Macro-F1	0.4728

Custom Feature 2: Character N-gram Featurizer

Location: `models.py`, class `CharNgramFeaturizer`

Description: Uses character-level n-grams (n=3,4) instead of word-level features. Character n-grams capture sub-word patterns such as morphological suffixes (e.g., "-ing", "-tion"), word fragments, and are more robust to spelling variations and rare words. For example, the trigram "tec" appears frequently in tech/science articles, while "spo" is common in sports. The feature vector is TF-normalized and L2-normalized.

Results (vocab_size=5000, lr=0.1, epochs=30):

Metric	Value
Train Accuracy	0.9697
Dev Accuracy	0.7318
Dev Macro-F1	0.4928

Q7b (1 point)

Performance Analysis:

- **TF-IDF** achieves Dev Accuracy 0.7306 (with lr=0.1, epochs=30), which is slightly lower than BoW (0.7459). TF-IDF down-weights common words and up-weights discriminative terms, but the L2 normalization reduces feature magnitudes, requiring a higher learning rate and more epochs to converge with SGD. The Macro-F1 (0.4728) is lower than accuracy, reflecting the class imbalance issue.
- **Character n-gram featurizer** achieves Dev Accuracy 0.7318 (with lr=0.1, epochs=30), comparable to TF-IDF. Character n-grams capture sub-word morphological patterns (e.g., "-ing", "-tion") and are robust to spelling variations, but miss higher-level semantic distinctions that whole words provide. The Macro-F1 (0.4928) is slightly better than TF-IDF,

suggesting character patterns provide more balanced class discrimination.

Task 4: A Better Evaluation Metric

Q8a (3 points)

Macro-F1 Implementation: Implemented in `utils.py`. For each class, compute precision, recall, and F1, then average across all classes.

Results for LR model (Q4):

Metric	Value
Dev Accuracy	0.7471
Dev Macro-F1	0.5137

Q8b (2 points)

Is F1 significantly different from accuracy?

Yes, the Macro-F1 (0.5137) is substantially lower than accuracy (0.7471), a difference of over 23%.

What does this tell us about the classifier?

This large discrepancy reveals that the classifier performs **very unevenly across classes**. The dataset is highly imbalanced (1000 world news vs. 50 sports), so accuracy is dominated by the majority class — getting "world news" right contributes heavily since it's ~60% of the data. Macro-F1 weights all classes equally, so poor performance on minority classes (especially sports with only 50 examples) severely hurts the score. The classifier likely under-predicts minority classes because predicting the majority class minimizes overall error, but this strategy fails for balanced evaluation metrics like Macro-F1.